

SOF 103 Week 11: Inheritance

Generated by Review Agent on 2026-07-02 06:36 UTC

Source: SOF 103-Chapter 11 - Inheritance2024_09.ppt

Classification confidence: 80%

1. Core Knowledge Outline

中文大纲:

- **面向对象编程 (OOP) 回顾**
- 什么是类 (Class) 和对象 (Object)?
- 封装 (Encapsulation) 的概念: 将数据和操作数据的方法捆绑在一起。
- **继承 (Inheritance) 的核心概念**
- **定义:** 一种机制, 允许一个类 (子类/派生类) 基于另一个类 (父类/基类) 来构建, 从而**复用、扩展和修改**父类的行为。
- **为什么重要?** 实现代码复用, 建立类之间的层次关系 (is-a 关系), 支持多态。
- **C++ 中的继承语法**
- 使用冒号 `:` 和访问说明符 `public`, `protected`, `private` 来声明继承关系。
- 示例: `class Derived : public Base { ... };`
- **继承中的访问控制**
- `public` 继承: 父类的 `public` 成员在子类中仍是 `public`, `protected` 成员仍是 `protected`。
- `protected` 继承: 父类的 `public` 和 `protected` 成员在子类中都变成 `protected`。
- `private` 继承: 父类的 `public` 和 `protected` 成员在子类中都变成 `private`。
- **构造函数和析构函数在继承中的行为**
- **构造顺序:** 先构造父类, 再构造子类。
- **析构顺序:** 先析构子类, 再析构父类。
- **构造函数调用:** 子类构造函数可以通过初始化列表显式调用父类的特定构造函数。
- **函数重写 (Function Overriding)**
- 子类可以重新定义 (重写) 父类中已有的成员函数。

- 这为实现**运行时多态** (Runtime Polymorphism) 奠定了基础。
- **多态 (Polymorphism) 与虚函数 (Virtual Functions)**
- **虚函数**: 使用 `virtual` 关键字声明的成员函数, 允许通过基类指针或引用来调用子类重写后的版本。
- **纯虚函数与抽象类**: `virtual void func() = 0;` 包含纯虚函数的类称为抽象类, 不能实例化, 只能作为接口使用。
- **多重继承 (Multiple Inheritance)**
- 一个子类可以继承自多个父类。
- 可能引发**菱形继承问题** (Diamond Problem) , C++ 通过**虚继承** (`virtual`) 来解决。

English Outline:

- **Review of Object-Oriented Programming (OOP)**
- What are Classes and Objects?
- The concept of Encapsulation: bundling data and methods that operate on that data.
- **Core Concept of Inheritance**
- **Definition**: A mechanism that allows a class (derived/subclass) to be based on another class (base/superclass), enabling the **reuse, extension, and modification** of the base class's behavior.
- **Why is it important?** Enables code reuse, establishes hierarchical relationships (is-a relationship) between classes, and supports polymorphism.
- **Inheritance Syntax in C++**
- Declared using a colon `:` and an access specifier (`public`, `protected`, `private`).
- Example: `class Derived : public Base { ... };`
- **Access Control in Inheritance**
- `public` inheritance: Public members of the base class remain public; protected members remain protected.
- `protected` inheritance: Public and protected members of the base class become protected.
- `private` inheritance: Public and protected members of the base class become private.
- **Constructor and Destructor Behavior in Inheritance**

- **Construction Order:** Base class constructor is called first, then the derived class constructor.
 - **Destruction Order:** Derived class destructor is called first, then the base class destructor.
 - **Constructor Invocation:** Derived class constructors can explicitly call a specific base class constructor using an initializer list.
 - **Function Overriding**
 - A derived class can redefine (override) a member function that already exists in the base class.
 - This is the foundation for **Runtime Polymorphism**.
 - **Polymorphism and Virtual Functions**
 - **Virtual Functions:** Member functions declared with the `virtual` keyword, allowing calls through a base class pointer or reference to invoke the overridden version in the derived class.
 - **Pure Virtual Functions & Abstract Classes:** `virtual void func() = 0;` A class with a pure virtual function is an abstract class and cannot be instantiated; it serves as an interface.
 - **Multiple Inheritance**
 - A derived class can inherit from more than one base class.
 - Can lead to the **Diamond Problem**, which C++ solves using **Virtual Inheritance** (`virtual`).
-

2. Key Concepts & Glossary

Term	Definition	Memory Anchor / Analogy
基类 / 父类 (Base Class / Parent Class)	被其他类继承的类。它定义了通用的属性和行为。 The class that is inherited from. It defines common attributes and behaviors.	类比：蓝图模板。就像汽车的设计蓝图，定义了所有汽车都有的基本功能（如方向盘、轮子）。 Analogy: Blueprint Template. Like the blueprint for a car, defining basic features all cars have (e.g., steering wheel, wheels).
派生类 / 子类 (Derived Class / Child Class)	从基类继承的类。它拥有基类的所有成员，并可以添加或修改自己的特定功能。 The class that inherits from a base class. It possesses all members of the base class and can add or modify its own specific features.	类比：具体车型。比如“SUV”是“汽车”的一个子类。它继承了汽车的所有功能，并增加了“四轮驱动”、“高底盘”等特性。 Analogy: Specific Car Model. An "SUV" is a subclass of "Car". It inherits all car functions and adds features like "4-wheel drive" and "high ground clearance".
继承 (Inheritance)	面向对象编程中，一个类基于另一个类来定义，从而复用代码并建立层次关系的机制。 An OOP mechanism where a class is defined based on another class to reuse code and establish a hierarchical relationship.	类比：遗传。孩子从父母那里继承特征（如眼睛颜色、身高），同时拥有自己独特的特征。 Analogy: Genetics. A child inherits traits from parents (e.g., eye color, height) while also having unique traits of their own.
函数重写 (Function Overriding)	在派生类中重新定义基类中已有的成员函数。 Redefining a member function that already exists in the base class within the derived class.	类比：定制化功能。基类定义了“启动引擎”的通用方法，但派生类（如“电动车”）可以重写这个方法，实现“静音启动”。 Analogy: Customized Functionality. The base class defines a generic "start engine" method, but a derived class (e.g.,

Term	Definition	Memory Anchor / Analogy
		"ElectricCar") can override it to implement a "silent start".
多态 (Polymorphism)	允许使用基类指针或引用来调用派生类重写后的函数的能力。这意味着“一个接口，多种实现”。 The ability to use a base class pointer or reference to call overridden functions in a derived class. It means "one interface, multiple implementations".	类比：通用遥控器。 你有一个“遥控器”（基类指针），按下“播放”按钮（虚函数）。对于“DVD播放器”和“流媒体盒子”（派生类），这个按钮会执行不同的操作。 Analogy: Universal Remote. You have a "remote" (base class pointer). Pressing the "play" button (virtual function) performs different actions for a "DVD player" vs. a "Streaming box" (derived classes).
虚函数 (Virtual Function)	在基类中使用 <code>virtual</code> 关键字声明的成员函数，它允许在运行时根据对象的实际类型来调用相应的函数版本。 A member function declared with the <code>virtual</code> keyword in the base class, allowing the appropriate function version to be called at runtime based on the object's actual type.	类比：占位符。 基类说“我会有一个播放功能，但具体怎么播放，由我的子类来决定”。这就是一个虚函数。 Analogy: Placeholder. The base class says, "I will have a 'play' function, but <i>how</i> to play is decided by my subclasses." This is a virtual function.
抽象类 (Abstract Class)	包含至少一个纯虚函数 (= 0) 的类。它不能被实例化，只能作为其他类的基类，用于定义接口。 A class that contains at least one pure virtual function (= 0). It cannot be instantiated and serves only as a base class to define an interface.	类比：概念车。 概念车展示了未来汽车的设计理念和功能，但它本身不能上路行驶（不能实例化）。它只是为后续的量产车（具体类）提供了一个模板。 Analogy: Concept Car. A concept car showcases design ideas and features but cannot be driven on the road (cannot be instantiated). It only provides a

Term	Definition	Memory Anchor / Analogy
		template for future production cars (concrete classes).
菱形继承问题 (Diamond Problem)	在多重继承中，当一个子类从两个继承自同一个基类的父类继承时，会导致结构上的歧义。 An ambiguity that arises in multiple inheritance when a derived class inherits from two parent classes that both inherit from the same base class.	类比：家族谱系混乱。想象一下，你的祖父（基类）有两个孩子（父类1和父类2），而你同时继承了这两个孩子的财产。那么，祖父的同一件传家宝，你继承了两份，这就产生了歧义。 Analogy: Confusing Family Tree. Imagine your grandfather (base class) has two children (parent 1 & 2), and you inherit from both. You would inherit the same heirloom from your grandfather twice, creating ambiguity.

3. Critical Takeaways

中文总结：

- 1. 继承的核心是 “is-a” 关系：**只有当派生类确实是基类的一种特殊类型时，才应该使用公有继承（`public inheritance`）。例如，“猫”是一种“动物”。不要为了代码复用而滥用继承。
- 2. 理解构造和析构的顺序：**在创建派生类对象时，总是先调用基类构造函数，再调用派生类构造函数。析构时顺序完全相反。这是面试和考试中的高频考点。
- 3. 多态是OOP的精髓：**通过基类指针或引用调用虚函数，可以实现“一个接口，多种行为”。这是实现灵活、可扩展软件的关键。记住，只有通过指针或引用调用虚函数才能实现多态，直接通过对象调用不行。
- 4. 区分函数重载 (Overloading) 和函数重写 (Overriding)：**重载发生在同一个作用域内（如同一个类），函数名相同但参数不同。重写发生在继承关系中，子类重新定义父类的虚函数，函数签名必须完全相同。

5. **抽象类作为接口**：包含纯虚函数的类就是抽象类，它定义了一个契约或接口。任何继承它的具体类都必须实现这个纯虚函数，否则它自己也会变成抽象类。

English Summary:

1. **The Core of Inheritance is the "is-a" Relationship**: Use `public` inheritance only when a derived class truly is a specialized type of the base class. For example, a "Cat" is an "Animal". Don't misuse inheritance just for code reuse.

2. **Understand Constructor/Destructor Order**: When creating a derived class object, the base class constructor is always called first, followed by the derived class constructor. Destruction is in the exact opposite order. This is a frequent exam and interview topic.

3. **Polymorphism is the Essence of OOP**: Calling virtual functions through a base class pointer or reference enables "one interface, many behaviors". This is key to building flexible and extensible software. Remember, polymorphism only works through pointers/references, not with direct objects.

4. **Distinguish Overloading from Overriding**: Overloading occurs within the same scope (e.g., the same class) with the same function name but different parameters. Overriding occurs in an inheritance relationship where a derived class redefines a base class's virtual function with the exact same signature.

5. **Abstract Classes as Interfaces**: A class with a pure virtual function is an abstract class, defining a contract or interface. Any concrete class inheriting from it must implement the pure virtual function, or it will also become abstract.

4. Detailed Notes (AI-Expanded)

4.1 继承的动机与概念 / Motivation and Concept of Inheritance

中文：

在软件开发中，我们经常会遇到多个类拥有共同属性和方法的情况。例如，一个大学信息系统可能有 `Student`（学生）和 `Professor`（教授）两个类。它们都有 `name`（姓名）、`age`（年龄）和 `address`（地址）等属性，以及 `eat()`（吃饭）和 `sleep()`

(睡觉) 等方法。如果我们为每个类都独立编写这些代码，就会产生大量的重复，导致维护困难。

继承就是为了解决这个问题而生的。它允许我们创建一个通用的**基类**（或父类），例如 `Person`（人），其中包含所有通用的属性和方法。然后，我们可以创建 `Student` 和 `Professor` 作为**派生类**（或子类），它们会自动**继承** `Person` 的所有成员。这样，我们只需要在 `Student` 和 `Professor` 中编写它们特有的代码（例如，`Student` 有 `studentID` 和 `study()`，`Professor` 有 `employeeID` 和 `teach()`）。

记忆锚点：可以把基类想象成一个“通用模板”，派生类则是基于这个模板的“定制化版本”。这极大地提高了代码的复用性、可维护性和可扩展性。

English:

In software development, we often encounter multiple classes that share common attributes and methods. For example, a university information system might have `Student` and `Professor` classes. Both have attributes like `name`, `age`, and `address`, and methods like `eat()` and `sleep()`. Writing this code independently for each class would lead to massive duplication and maintenance difficulties.

Inheritance was created to solve this problem. It allows us to create a generic **base class** (or parent class), e.g., `Person`, containing all the common attributes and methods. Then, we can create `Student` and `Professor` as **derived classes** (or child classes), which automatically **inherit** all members of `Person`. We only need to write the code specific to `Student` and `Professor` in their respective classes (e.g., `Student` has `studentID` and `study()`, `Professor` has `employeeID` and `teach()`).

Memory Anchor: Think of the base class as a "generic template" and derived classes as "customized versions" built upon it. This drastically improves code reusability, maintainability, and extensibility.

4.2 C++ 继承语法与访问控制 / C++ Inheritance Syntax and Access Control

中文:

在 C++ 中，继承的语法如下：

```
class Base {
public:
    int publicVar;
protected:
    int protectedVar;
private:
    int privateVar; // 子类无法直接访问
};

class Derived : public Base {
    // ...
};
```

这里的 `public` 是**访问说明符**，它决定了基类成员在派生类中的访问权限。有三种继承方式：

1. **公有继承 (public)**：最常用。基类的 `public` 成员在派生类中仍然是 `public`，`protected` 成员仍然是 `protected`。基类的 `private` 成员在派生类中不可直接访问，但可以通过基类的公有或保护成员函数间接访问。这体现了 “is-a” 关系。
2. **保护继承 (protected)**：基类的 `public` 和 `protected` 成员在派生类中都变成 `protected`。这意味着这些成员在派生类内部可以访问，但对外部代码（非派生类的对象）是不可见的。
3. **私有继承 (private)**：基类的 `public` 和 `protected` 成员在派生类中都变成 `private`。这是最严格的继承方式，通常用于 “implemented-in-terms-of”（用...来实现）的关系，而不是 “is-a” 关系。

English:

In C++, the syntax for inheritance is:

```
class Base {
public:
    int publicVar;
protected:
    int protectedVar;
private:
    int privateVar; // Not directly accessible by derived class
};
```

```
class Derived : public Base {  
    // ...  
};
```

The `public` here is an **access specifier** that determines how the base class members are accessed in the derived class. There are three types:

1. **Public Inheritance (`public`)**: The most common. Base class `public` members remain `public`, and `protected` members remain `protected` in the derived class. Base class `private` members are not directly accessible but can be accessed indirectly via public or protected member functions of the base. This models the "is-a" relationship.
2. **Protected Inheritance (`protected`)**: Both `public` and `protected` members of the base class become `protected` in the derived class. This means they are accessible within the derived class but not to external code (non-derived class objects).
3. **Private Inheritance (`private`)**: Both `public`